

[SFTW] Software Reengineering
- Odoo Community - [GitHub](#)

Final Report - 12/06/2025

Ilias Sarikakis
Pablo de Vicente Abad

Table of Contents

1. Introduction.....	3
2. Background.....	3
2.1 Odoo.....	3
2.2 First contact.....	4
2.3 Hands-on approach.....	5
2.4 Tool Usage.....	6
2.4.1 Metrics and Visualisation.....	6
2.4.2 Code duplication analysis.....	9
2.4.3 Refactoring assistants.....	10
2.4.4 Test coverage.....	11
2.4.5 Mining repositories.....	12
2.4.6 Database Overview.....	13
3. Scope and goals.....	15
3.1 Loyalty module Overview.....	16
3.2 Our proposal.....	18
4. Reengineering efforts.....	19
4.1 Our approach.....	19
4.2 Considered Reengineering Patterns (OORP).....	20
4.3 Refactoring efforts.....	22
4.4 Changes and new additions.....	26
4.5 Integration with existing database.....	30
5. Conclusions.....	31

1. Introduction

This report presents a detailed account of our thought process, methodology, and final solution in reengineering a critical segment of the Odoo codebase. Our focus centers on addressing a significant limitation identified within the *loyalty* and *sale_loyalty* modules, which are responsible for awarding loyalty points to customers and redeeming those points as discounts on their respective orders.

This project was carried out as part of the **Software Reengineering** course at the University of Antwerp, following the course's specified format and evaluation criteria.

2. Background

Before diving into the reengineering efforts, it is essential to provide an overview of the Odoo codebase. This context will help clarify the system's architectural structure and the development methodologies adopted by its developers.



2.1 Odoo

Odoo is an open-source suite of business applications designed to address a variety of enterprise needs, including Customer Relationship Management (CRM), eCommerce, accounting, project management, and more.

According to the official [Odoo website](#), the platform is built around a modular architecture. Each business application is implemented as an independent module, designed to handle specific functionalities such as invoicing, inventory management, or human resources.

These modules operate under the core *Odoo framework*, which provides the foundational infrastructure and shared services required by all modules. Notably, the modular architecture allows for independent interaction between modules, enabling a high degree of flexibility and customization depending on the business requirements.

2.2 First contact

We started off by forking the community edition (version 18.0) into a local [GitHub repository](#). Following the official setup documentation provided by Odoo, we were able to successfully deploy the project in a local development environment. The successful execution of the platform is illustrated in [Figure 1](#).

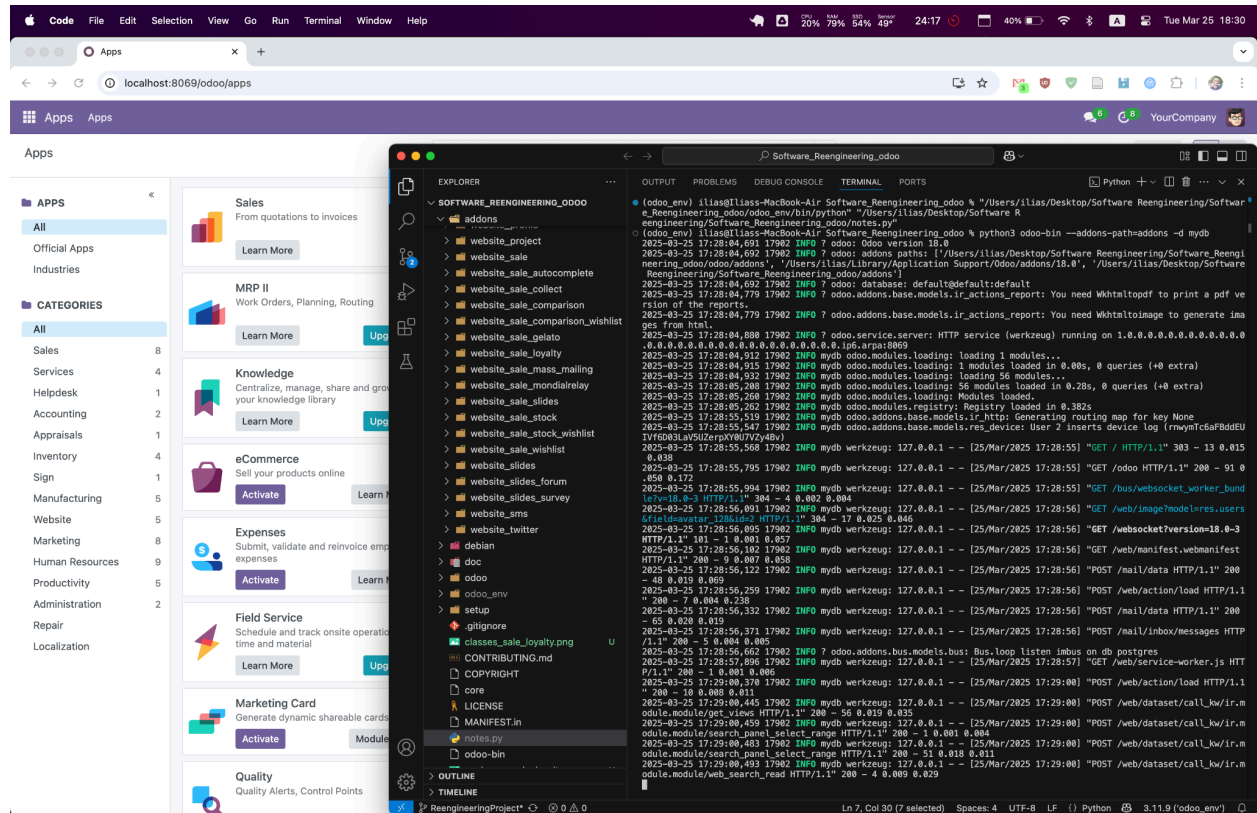


Figure 1. Odoo community edition running locally on our system

As highlighted in *Object-Oriented Reengineering Patterns* (OORP, p.36), reengineering frequently begins with a fundamental question: **Where do I start?** (OORP, p.37). Given that the Odoo project is hosted on a public GitHub repository, we applied a combination of practical, hands-on exploration and automated reengineering tools to examine the codebase comprehensively.

We began by combining hands-on application with analytical tools to better understand the complexity of the project. **Sections 2.3 and 2.4** provide a detailed overview of the methodologies used. These methodologies are examined from both organizational and functional perspectives, as well as the intermediate considerations that shaped the definition of our reengineering scope and objectives.

2.3 Hands-on approach

The objective was to explore the software from a usability perspective, identifying both its capabilities and its limitations through direct engagement with the system.

Our strategy involved examining specific components of the ecosystem that appeared underdeveloped or lacking in essential features. In particular, we focused on detecting functionalities typically expected in comparable enterprise solutions but either absent or only partially realized within Odoo.

Example:

In a standard payment process, users typically expect multiple payment options, such as credit/debit cards and PayPal. If one of these options is unavailable, we would investigate the feasibility of implementing the missing functionality and determine the extent to which it could be integrated.

Guided by the project description and our initial observations, we focused our attention on the **Loyalty** module in Odoo. We found that this module could be activated via the Odoo interface within our local development environment, as shown in [Figure 2](#).

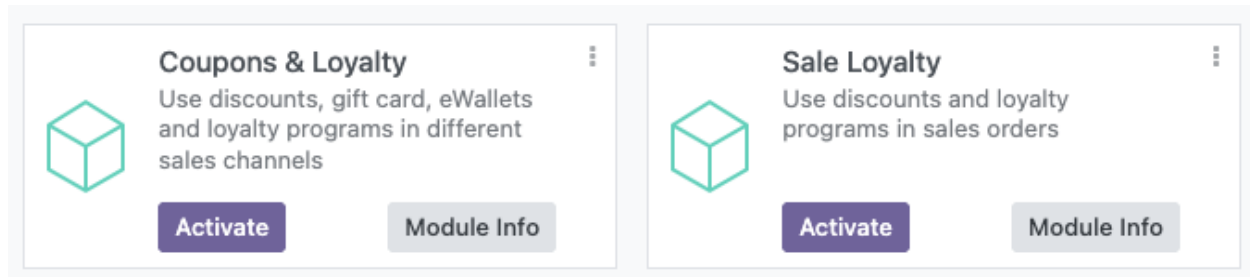


Figure 2. The core “Loyalty” module (Coupons & Loyalty) and “Sale Loyalty” modules ready to be activated

Activating the module enabled us to interact with its functionality directly through the Odoo web interface. This hands-on exploration provided a high-level overview of how the module operates, without the need to immediately engage with the underlying source code.

A detailed analysis of our findings, including the module's behavior and how it integrates within the broader Odoo framework, is presented in **Section 3**.

2.4 Tool Usage

While our initial exploration of the Odoo loyalty module focused on understanding its functionality through the user interface, we complemented this approach with a structured, tool-assisted analysis of the underlying codebase. Using a range of software analysis tools—including refactoring assistants and code duplication detectors—we were able to systematically examine the architecture and internal logic of the system.

Our tool-based investigation focused on five key areas:

1. **Code Metrics and Visualization** – We observed visual representations of the codebase to better understand its structure and complexity.
2. **Code Duplication Analysis** – We identified redundant code segments that could benefit from consolidation and simplification.
3. **Refactoring Opportunities** – We pinpointed areas where the code exhibited signs of poor maintainability, such as long methods or high complexity, indicating the need for refactoring.
4. **Test Coverage Evaluation** – We assessed the extent and effectiveness of the existing test suite.
5. **Repository Mining** – We conducted a brief analysis of the version control history to identify key contributors and maintainers. This simulated a real-world scenario in which we, as external consultants or new team members, would need to contact knowledgeable stakeholders for further clarification or collaboration.

2.4.1 Metrics and Visualisation

The principal tool we used throughout this process was **CodeScene**, a comprehensive software analysis platform. CodeScene proved instrumental not only for collecting various software metrics and visualizing project structure but also for guiding our refactoring decisions and conducting repository mining.

Visualization of the Codebase

CodeScene enables us to quickly map the extensive Odoo codebase into a 2D space. It clusters project directories and files, presenting the information in a clear and accessible format while at the same time allowing for various views and filter options to only show the most relevant information for our specific use case. This visualization tool provides multiple options to explore the structure and dynamics of the code.

Identifying Hotspots and Code Health

One of the primary features of CodeScene is its ability to display different viewpoints of the code clusters, as seen in [Figure 3](#). For example, it highlights areas with high development activity (commonly referred to as hotspots) by tracking the frequency of recent commits to specific files. It also provides an assessment of the overall code health for each file, along with insights into which files are routinely modified together during commits.

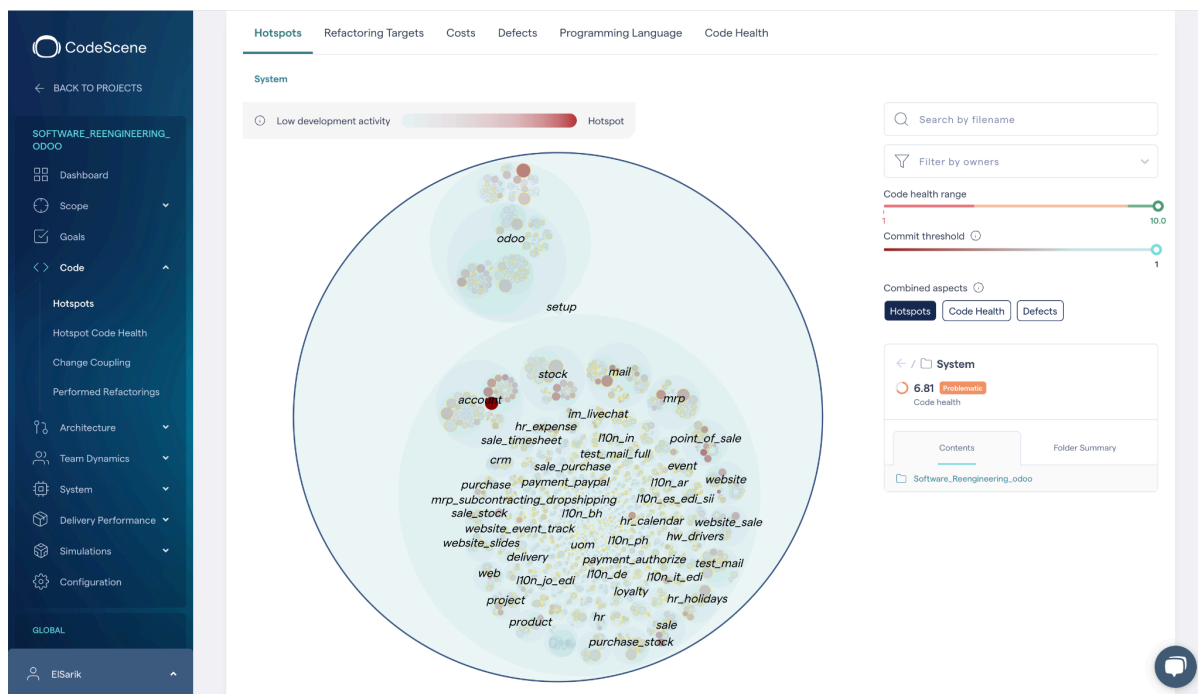


Figure 3. Odoo codebase visualization in CodeScene showcasing the code's hotspots

In-Depth X-Ray Analysis

CodeScene further offers an X-Ray view, which allows us to zoom in on individual files to pinpoint specific issues. This view, depicted in [Figure 4](#), details complex functions and methods, highlights areas with deep nested complexity, and flags parts of the code that exhibit rule violations or other problematic patterns.

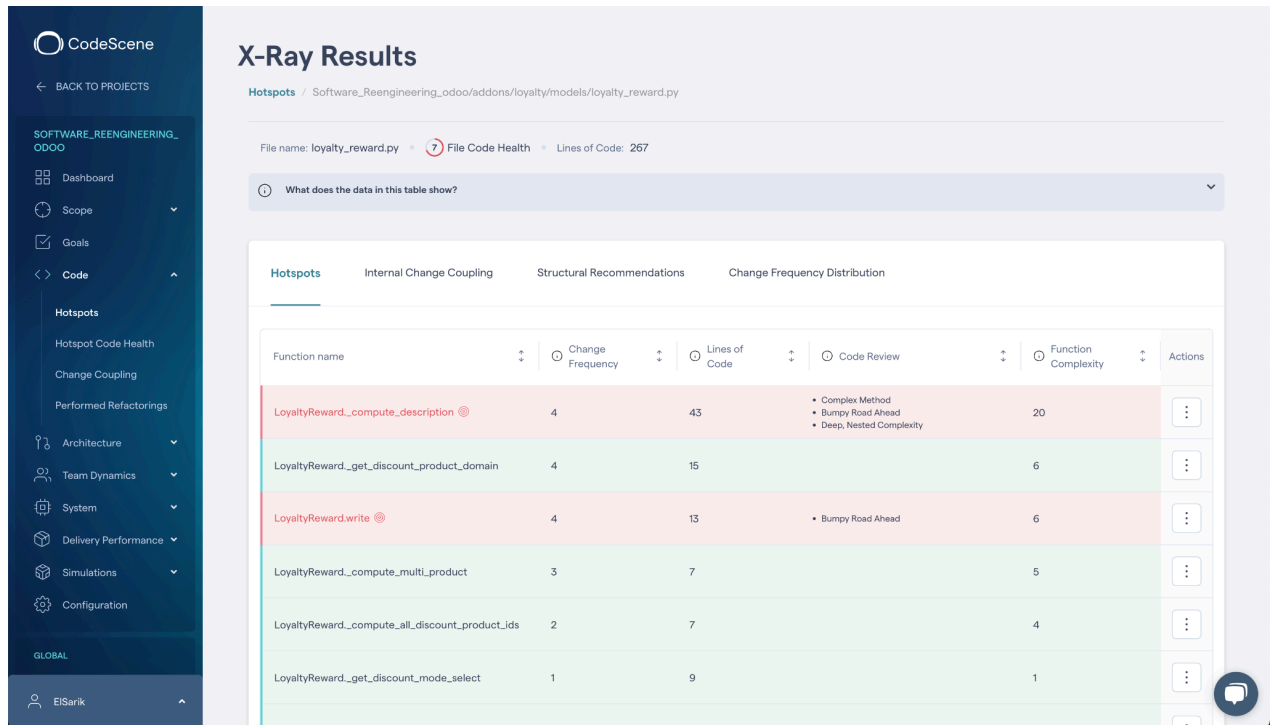


Figure 4. X-Ray results view in CodeScene

By providing these comprehensive analysis tools, CodeScene facilitates a deeper understanding of the codebase and supports efficient identification of refactoring targets, thereby enhancing our overall reengineering efforts.

2.4.2 Code duplication analysis

In this section, our objective was not immediate problem-solving but rather comprehensive investigation. Our approach aligned with the principle stated in the book *If It Ain't Broke, Don't Fix It* (ORP, p.35). We dove deep into the software, aiming to improve our understanding.

PMD and CPD

For our code duplication analysis, we initially considered employing the [PMD tool](#), a multi-language static code analyzer that identifies common programming flaws (unused variables, empty catch blocks, and unnecessary object creation) in languages like Java, JavaScript, Apex, and several others. However, as noted by the tool's authors, PMD does not support Python.

Recognizing our need for a Python-compatible solution, we turned to CPD (Copy-Paste Detector), which is included as part of PMD. CPD is designed to uncover duplicated code across a broader range of languages, including C/C++, PLSQL, and Python. This capability made it the ideal choice for our analysis.

Scope of Analysis

Due to the immense size of the code base that would need to be reviewed, we adopted a pre-filtering strategy to focus our efforts on the *loyalty* module so that we would be on par with our hands-on approach. This targeted approach meant that, while we might have overlooked some sections, we could more efficiently evaluate the outputs from areas that appeared the most critical based on our hands-on knowledge of the codebase.

It is important to note that while CPD is effective at identifying duplicate code, it does not provide insights into deeper semantic issues related to code quality or design. Its role is primarily as a diagnostic tool to pinpoint areas that may benefit from further manual inspection and refactoring.

2.4.3 Refactoring assistants

CodeScene offers a range of valuable insights for identifying potential refactoring targets. It highlights specific areas of the code that exhibit high development activity combined with low overall code health, marking them as prime candidates for refactoring. Targeting these areas can provide a significant return on investment, as the frequency of modifications indicates their strategic importance for the company.

Furthermore, CodeScene provides a comprehensive analysis of the health status for individual files, complete with clear explanations of why certain files are flagged as problematic. These explanations come in the form of warnings and best practice rule violations as seen in [Figure 5](#).

For instance, warnings might include issues like overall code complexity, overly large methods, or an excessive number of function arguments. Similarly, rule violations can include factors such as low cohesion, deeply nested complexity, or "bumpy road" code—characterized by nested conditional logic within a single function.

The use of CodeScene for analysis is essential to understanding the project, and we were impressed by the wealth of information provided by this user-friendly tool—unlike others we tested. We confidently recommend CodeScene for both analysis and refactoring assistance.

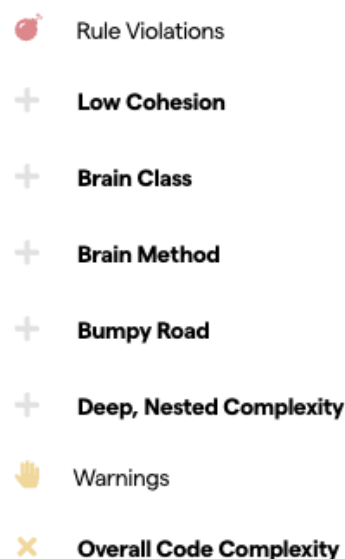


Figure 5. CodeScene output for a sample file, highlighting various issues such as low cohesion, deeply nested complexity, and basic rule violations. See Appendix 1 for the complete output.

2.4.4 Test coverage

One of the core reasons testing is so essential is well captured by ***Tests are your life insurance!*** (OORP, p.149), and further reinforced by ***Write Tests to Enable Evolution*** (OORP, p.153). Without proper tests, we lack a solid foundation for safely making changes or refactoring the system.

The Odoo project already included a wide range of tests. This is great, as we could use the existing test suite to establish a baseline for the state of the system at the time. If any changes we implemented were to cause some tests to fail, we would have a clear indication of where the failure occurred.

To complement our analysis of the test coverage, we utilized [Coverage.py](#), a widely-used tool for measuring code coverage in Python projects. By applying this tool to the Odoo codebase, we were able to gain a clearer understanding of the extent to which the existing code was tested. The results generated by Coverage.py were uploaded to our GitHub repository for transparency and future reference.

These baseline coverage metrics served as an important reference point, enabling us to assess the impact of our reengineering efforts. **By comparing pre- and post-refactoring coverage levels, we could evaluate whether our changes maintained or improved the system's testability and overall reliability.**

2.4.5 Mining repositories

Repository mining involves analyzing the evolution and contributor history of a codebase to identify hotspots, key maintainers, and areas of complexity.

Using CodeScene, we could quickly map parts of the codebase to specific developers, information which becomes particularly valuable when planning reengineering efforts. Instead of manually exploring the source code to understand how a module works or how it interacts with others, we could consult the person most familiar with it. For instance, if we were evaluating the `sale_loyalty` addon, we would reach out to William Braeckman, who appears to be its main maintainer as seen in [Figure 6](#).

Such strategies streamline collaboration and reduce time spent on code. In a production environment, this approach would help focus discussions, gather the necessary context quickly, and ensure that reengineering proposals are grounded in both code and contributor knowledge. In addition to CodeScene, other tools such as GitHub's own contributor insights, Git log analysis, and metrics like code commit frequency could further inform our decisions and help prioritize targets for refactoring.

For our specific efforts, this aspect might not be immediately relevant, however it shows promise for larger-scale refactoring and reengineering projects.

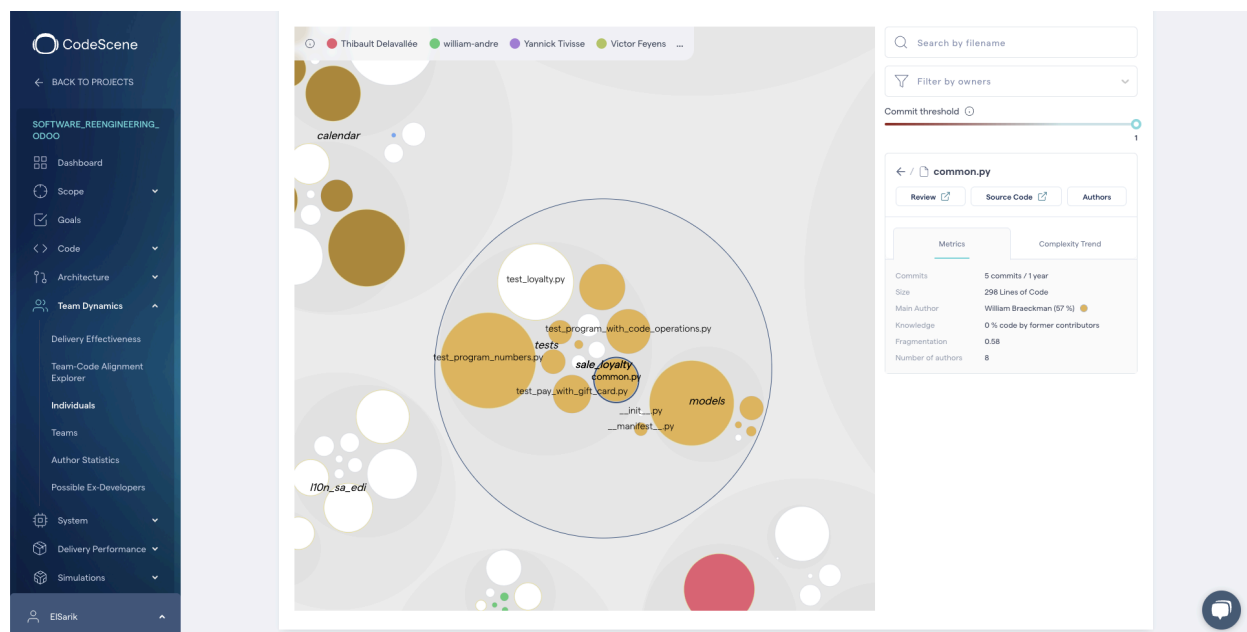


Figure 6. CodeScenes view for mining repository information

2.4.6 Database Overview

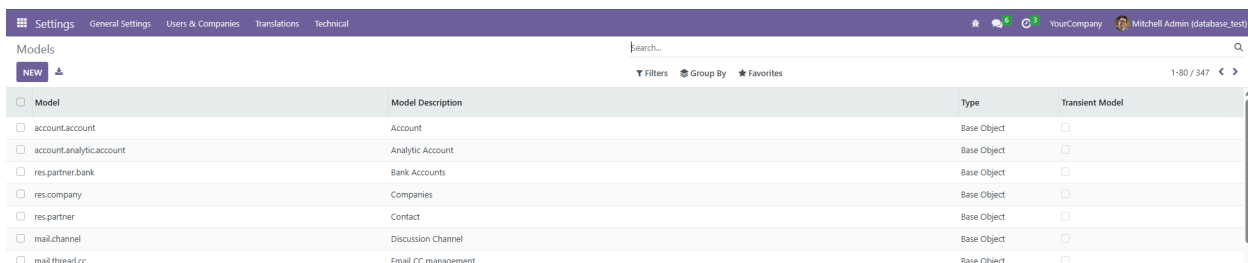
We wanted to gain a deeper understanding of how Odoo's database is structured. To do this, we took advantage of both the Odoo web interface and external tools.

Exploring Odoo's Database Structure

Within the Odoo web server, you can inspect the database structure by navigating through:

Settings > Technical > Database Structure > Models

This path lets you examine various models, their fields, and how they interrelate, giving a clearer picture of how the business logic is organized and how data flows within the system.



Model	Model Description	Type	Transient Model
account.account	Account	Base Object	☐
account.analytic.account	Analytic Account	Base Object	☐
res.partner.bank	Bank Accounts	Base Object	☐
res.company	Companies	Base Object	☐
res.partner	Contact	Base Object	☐
mail.channel	Discussion Channel	Base Object	☐
mail.thread.cc	Email CC management	Base Object	☐

Figure 7. Database structure as seen from Odoo's server

Leveraging PgAdmin for PostgreSQL Analysis

To further investigate the underlying PostgreSQL database, we installed [PgAdmin](#). This tool allows us to browse and query tables and schemas, apart from visualizing relationships between tables and review data integrity.

pgAdmin4

FileObjectToolsEditViewWindowHelp

Welcome

mydb3/postgres@odoo_testing X

mydb3/postgres@odoo_testing

No limit

QueryQuery History

Scratch Pad X

1SELECT * from res_users;

Data OutputMessagesNotifications

Showing rows: 1 to 7 | Page No: 1 of 1

	id [PK] integer	company_id integer	partner_id integer	active boolean	create_date timestamp without time zone	login character varying	password character varying	action_id integer	create_uid integer	write_uid integer
1	4	1	4	false	2025-03-25 17:13:45.500774	public	Spokd2-sha51256000005A8DYW0u.McY4Z6vY.prtzsg5eFrc5xhP6v.p27.geTSPb8PthREqx317quXefky3.IDu5gmyELUsYxMNO4Xv9hXkLQ3N5y3j04QvA	[null]	1	1
2	1	1	2	false	2025-03-25 17:13:45.500774	...system...	[null]	[null]	[null]	1
3	3	1	5	false	2025-03-25 17:13:45.500774	default	[null]	[null]	[null]	1
4	5	1	6	false	2025-03-25 17:13:45.500774	portaltemplate	[null]	[null]	[null]	1
5	6	1	7	true	2025-03-25 17:13:45.500774	demo	Spokd2-sha51256000005NGa8967FyKfJAJY5.mduwS3.FiltsTP57b./wDfLPteVZgpfPdm7S80.HdVapTSXZj0ZRAA9bApz2FHeEE1nF04Z7i7CY.TMbuAXw	[null]	[null]	1
6	7	1	8	true	2025-03-25 17:13:45.500774	portal	Spokd2-sha51256000005WouR817737xAgBg020zow5N0BvmmED5c.66Fh9lgeX.MGRhUJZT185gEauc2k74868ROzWimsdWELI.nyDkaFJku7H7A7A7uCaMR...	[null]	[null]	1
7	2	1	3	true	2025-03-25 17:13:45.500774	admin	Spokd2-sha51256000005713v/T9nT0K9J0TohWm80SkYjsAYvfhTfXk62wJXKawFNetZL7.2CyfrnMfYURLMcZBgHhU/UsQW.J0MK9QD4dbccxUTY1Yvgbs57GA	[null]	[null]	1

Figure 8. Database structure as seen from PgAdmin

Exploring Product and Order Tables in Odoo

To deepen our understanding of the Odoo system and its internal structure, we identified several database tables that contain relevant information about products and orders:

product_product: This table stores detailed information about individual products. It's important to note that product variants (e.g., different materials such as wood or iron) are stored as separate entries.

product_tag: Table that defines the available tags used to categorize products. Tags are a core component of the functionality we plan to leverage for differentiating between product types.

product_tag_product_product_rel and **product_tag_product_template_rel**: These relationship tables link products to their associated tags, enabling flexible categorization and filtering.

While these aspects are primarily managed internally by the Odoo system, familiarizing ourselves with the underlying data structure is valuable for debugging and troubleshooting in future development work.

Additional Resources

To enhance our understanding of PostgreSQL database architecture, we consulted several external resources. Although not all references are listed here for brevity, one particularly useful example is a [Comprehensive Guide to PostgreSQL Database Structure](#)

This guide explains key objects within PostgreSQL and how they work together, providing a framework that helped us understand what we observed in both Odoo and PgAdmin. By combining these methods and resources, we built a foundation for understanding the complex interplay between Odoo's ORM-driven model layer and the PostgreSQL backend.

3. Scope and goals

This section provides a comprehensive overview of the loyalty module in Odoo, including its intended functionality and operational mechanics. Additionally, it outlines the specific scope of our reengineering effort, detailing the objectives we set out to achieve and the rationale behind our proposed improvements.

Building on our initial investigation, described in **Sections 2.3 and 2.4**, we identified several notable limitations in the current implementation and core functionality of the module. A more detailed analysis using CodeScene, illustrated in [Figure 9](#), revealed problematic areas within the module's codebase, reinforcing its suitability as a primary target for reengineering within the scope of our project.

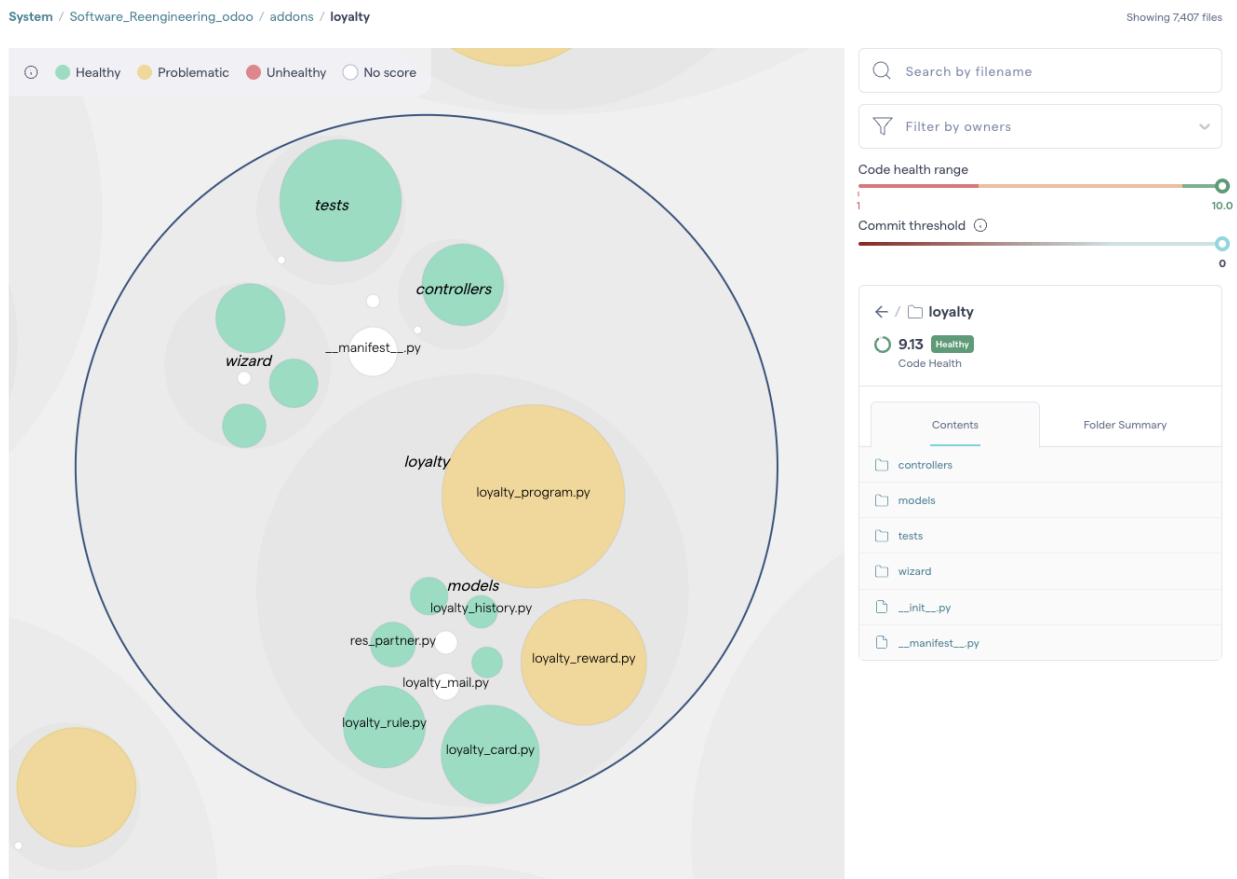


Figure 9. Code Health Overview from Odoo using CodeScene

3.1 Loyalty module Overview

The *Loyalty* module in the Odoo suite serves as a complementary extension to the core *Sales* functionality, which handles the buying / selling of the goods and services the company provides. Its main purpose is to reward customers based on their purchases and allow them to redeem these rewards in future transactions.

The company using Odoo (referred to as the provider) can configure rules that grant a set number of loyalty points to customers depending on the items they buy. These rules can be linked to specific products or entire product categories. For example, purchasing products from Category A might earn the customer 10 points, while products from Category B might earn 20 points, and so on.

Customers gradually accumulate these loyalty points on their loyalty cards and can later redeem them for discounts on future orders. The provider can also define rules specifying which products or product categories are eligible for discounts and the discount amounts.

As illustrated in [Figure 10](#), the UML diagram shows the interaction between the Loyalty module and the Sales module. The customer, referred to as a Partner in the diagram, holds a loyalty card that tracks all accumulated points from past orders. This card is associated with a loyalty program that defines both how points are awarded and how they can be redeemed. When a customer places an order by selecting products, they have the option to either earn loyalty points or redeem existing points to receive discounts.

As shown in the UML diagram, the fields `product_category_id` (in the `loyalty.rule` class) and `discount_product_category_id` (in the `loyalty.reward` class) are defined as `Many2one` relationships. This implies that each rule can be linked to only one product category, which unintentionally limits the capabilities of the entire module.

Consider the following scenario: we want to award 20 loyalty points per order when a customer purchases products from both Category A and Category B. Due to the current implementation, we must create two separate rules, each granting 10 points per order, one for Category A and another for Category B. This setup is illustrated in [Figure 11](#) under the "Conditional Rules" section.

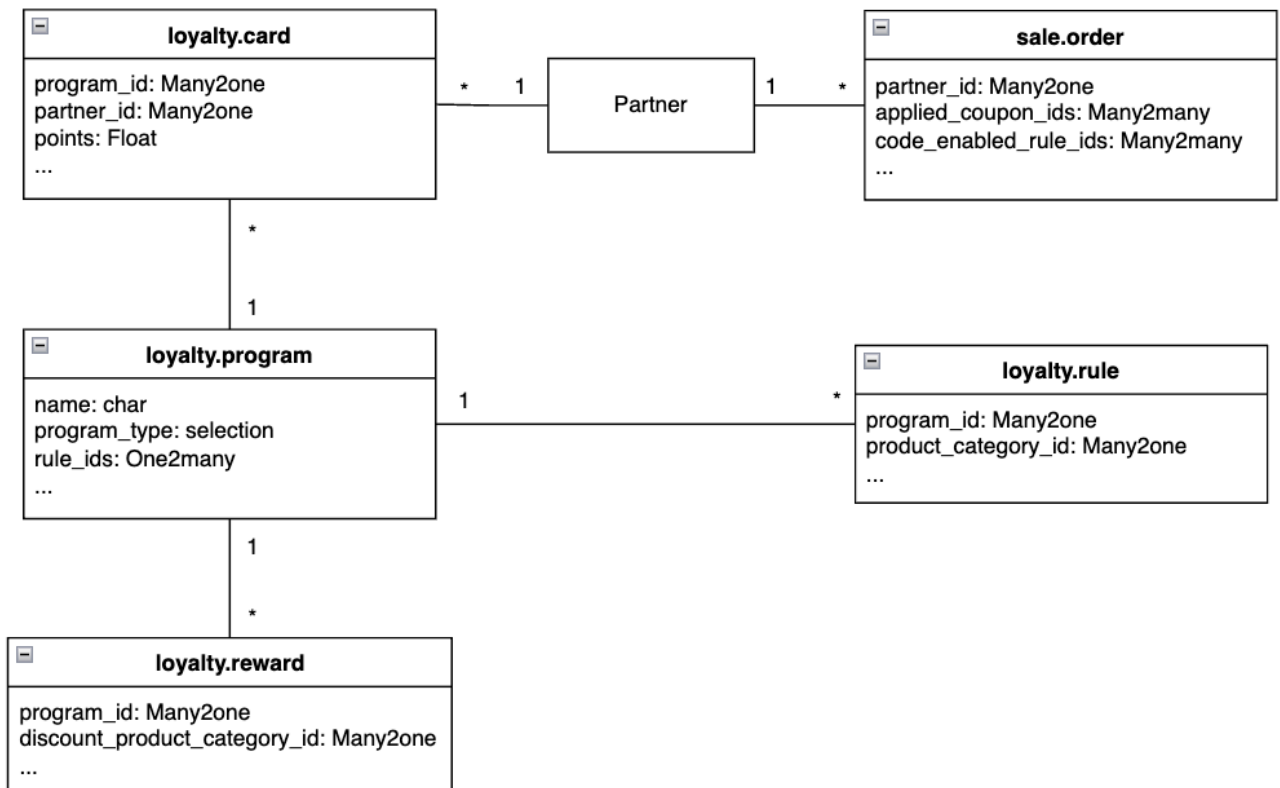


Figure 10. UML diagram depicting how the loyalty module is structured and interacts with the customers (partner) and the sales module.

Program Name

AB

Program Type

Loyalty Cards

When customers make an order, they accumulate points they can exchange for rewards on the current order or on a future one.

Start Date ?

End date ?

Currency

USD

Limit Usage

☐

Points Unit

Loyalty point(s)

Available On

☒ Sales

Rules & Rewards

Communications

Conditional rules

Add

Among:

A

Grant

10.00 Loyalty point(s) per order

Among:

B

Grant

10.00 Loyalty point(s) per order

Rewards

Add

5.00% discount on your order

In exchange of

200.00 Loyalty point(s)

Figure 11. Example setup of how to award customers with 20 loyalty points for orders with Category A and Category B products

This implementation produces the expected outcome: when an order includes products from both Category A and Category B, the customer is awarded a total of 20 loyalty points. In contrast, if the order contains products from only one of these categories, the customer receives 10 points. However, this behavior is fixed and cannot be modified by the provider, as the current design only allows each rule to be associated with a single product category. This means that with the current approach, the customer will always receive some amount of loyalty points as long as their order contains either Category A or Category B products, which may not be the intended behavior.

As a result, the existing structure of the loyalty module limits its flexibility. In particular, it does not support the creation of rules that reward customers based on the presence of products from **multiple categories** within the same order. Introducing support for such combinations would enable more advanced and strategic reward schemes. For example, businesses could offer additional points when customers purchase complementary products - such as a table and a chair together - thereby encouraging bundled purchases and increasing overall sales.

3.2 Our proposal

In this project our goal is to overhaul Odoo's loyalty module by addressing the key limitations of the current implementation. Specifically we want to allow for multiple product categories to be selected within the loyalty rules. Furthermore we want to enhance the rules capabilities even further by allowing the provider to choose whether the loyalty points will be applied when **either** or **both** of the selected product categories exist in the order. This addition could allow for more possible combinations with regards to how the loyalty points are being awarded.

Based on the previous example, and the already established rules, orders with both A and B will hand out 20 points. With our proposal, we could create another rule, which gives 5 points in orders where **both** A and B categories are present (while still handing out the 20 points from before). By doing this, the customer is awarded 25 points in total, solely by the fact that they purchased an item from both categories. Alternatively, a rule could be created that awards 20 loyalty points **only** when both A and B categories are present and 0 in any other case.

Another benefit of our proposal is that the rules become easier to maintain, as it is now possible to have a rule which states that if a product of **either** A or B exists in the order, the user is rewarded 20 points, essentially combining the logic of 2 rules into 1.

In section 4.3 of the report we go into more detail regarding how we implemented our proposal in the existing codebase.

4. Reengineering efforts

Here we will dive into the approach that we followed for our project, the reengineering patterns we considered and how they benefited us, and we will analyze in detail how our proposal from the previous section was implemented and integrated with the existing codebase.

4.1 Our approach

The majority of this project involved evaluating and understanding the inner workings of the Odoo platform rather than extensive code development. Research and analysis took up the bulk of our time, with implementation being a smaller - but still important - part of the overall effort.

To facilitate collaboration, we established a GitHub repository early on. This allowed all team members to access the latest version of the code, contribute updates, and reflect on the project documentation. We also maintained shared documentation to coordinate work on report drafts and technical notes.

Due to differing schedules, we often worked asynchronously. However, communication remained fluid and effective. To stay aligned, we agreed to hold weekly meetings. These meetings served several purposes:

- Sharing **progress updates** to ensure we remained on the same page,
- **Updating our task list** with current issues and brainstorming potential solutions,
- **Outlining a short-term plan** - typically a two-week roadmap with clear goals for the upcoming week.

These meetings were held regularly throughout the semester, with only minor exceptions due to scheduling conflicts or other academic commitments.

4.2 Considered Reengineering Patterns (OORP)

Throughout the course of the project, we employed a variety of software reengineering techniques and design patterns, many of which are derived from established practices presented in ***Object-Oriented Reengineering Patterns*** by Serge Demeyer et al. These patterns served as a guiding framework across multiple phases of our work—including requirements analysis, implementation, and testing. While we did not adhere strictly to the process model illustrated in [Figure 12](#), the team agrees that our workflow closely resembled its overarching rationale.



Figure 12. Principles and guidelines to set and maintain direction in our reengineering project.

Appoint a Navigator

Early in the project, Ilias Sarikakis was designated as the Navigator. This role involved maintaining a high-level understanding of the project's direction, guiding architectural decisions, and ensuring that design choices aligned with the overall goals. Having a clear navigator helped balance the detail-oriented nature of development with a broader strategic perspective.

Speak to the Round Table

We held regular weekly meetings throughout the project. These meetings gave us the chance to share updates, reflect on problems, set objectives, and coordinate on tasks. This rhythm helped us maintain a good balance of staying goal-oriented while also leaving room for team discussion and adjustment. It promoted ongoing communication without overwhelming our schedules and contributed significantly to the project's steady progress.

If It Ain't Broke, Don't Fix It

One of the key principles we followed was minimizing unnecessary changes to the codebase. Although we did refactor parts of the system to support maintainability, we were mindful not to over-engineer or rewrite working components unless there was a clear justification. This aligns with the pattern's focus on minimizing risk and effort by preserving proven functionality. Even in cases where deeper refactoring seemed tempting, we reminded ourselves to be as non-intrusive as possible and only intervene where it added value - considering both future maintainability and feature expansion.

Keep It Simple

On the same note as the previous principle, while analyzing the problem and the Odoo architecture, we encountered numerous opportunities to expand the project's scope by adding broader features or reworking related systems. However, we deliberately chose to stay focused on our original, well-defined task, described in detail in **Section 3.2**. This allowed the team to deliver a working solution within scope and time constraints, while also laying the groundwork for potential future enhancements.

Test Early and Often

Though not a named pattern in the book, we followed a fundamental reengineering best practice: regression testing throughout the process. We consistently ran the existing test suite before and after changes, and supplemented it with new tests for any added functionality. This ensured that the core system remained stable and that our changes were safe and verifiable. Furthermore, we also manually tested our changes through Odoo's web interface, which provided immediate feedback and also an overview of how the user would interact with our changes.

These patterns helped us stay grounded in solid reengineering principles, enabling us to improve the system responsibly without overcomplicating the solution or introducing unnecessary risks.

4.3 Refactoring efforts

Before beginning the refactoring process, the first step was to figure out which parts of the loyalty module needed the most work. As previously mentioned in **Section 2.4.1**, we used the CodeScene tool to perform this analysis and identify these refactoring targets.

Function name	Change Frequency	Lines of Code	Code Review	Function Complexity
LoyaltyReward.compute_description ☹	4	43	- Complex Method - Bumpy Road Ahead - Deep, Nested Complexity	20
LoyaltyReward.get_discount_product_domain	4	15		6
LoyaltyReward.write ☹	4	13	Bumpy Road Ahead	6

Figure 13. Analysis results of the Initial state of Loyalty_reward file

[Figure 13](#) shows an example of the CodeScene analysis when examining a module. In particular, the `Loyalty_reward.py` script stood out, with several functions flagged as needing attention.

Function name	Change Frequency	Lines of Code	Code Review	Function Complexity
LoyaltyReward.compute_description ☹	5	43	- Complex Method - Bumpy Road Ahead	13
LoyaltyReward.write	5	13		3

Figure 14. Analysis results after the write method was modified

We began by simplifying one of the more complex functions, focusing especially on the `write` method. As shown in [Figure 14](#), this cleanup helped reduce complexity and made the function easier to follow. While there's still room to break it down further, we decided to keep a balance between splitting it into smaller parts and maintaining a straightforward, readable flow. Since the function is under 70 lines, it doesn't qualify as particularly large, which supports this decision.

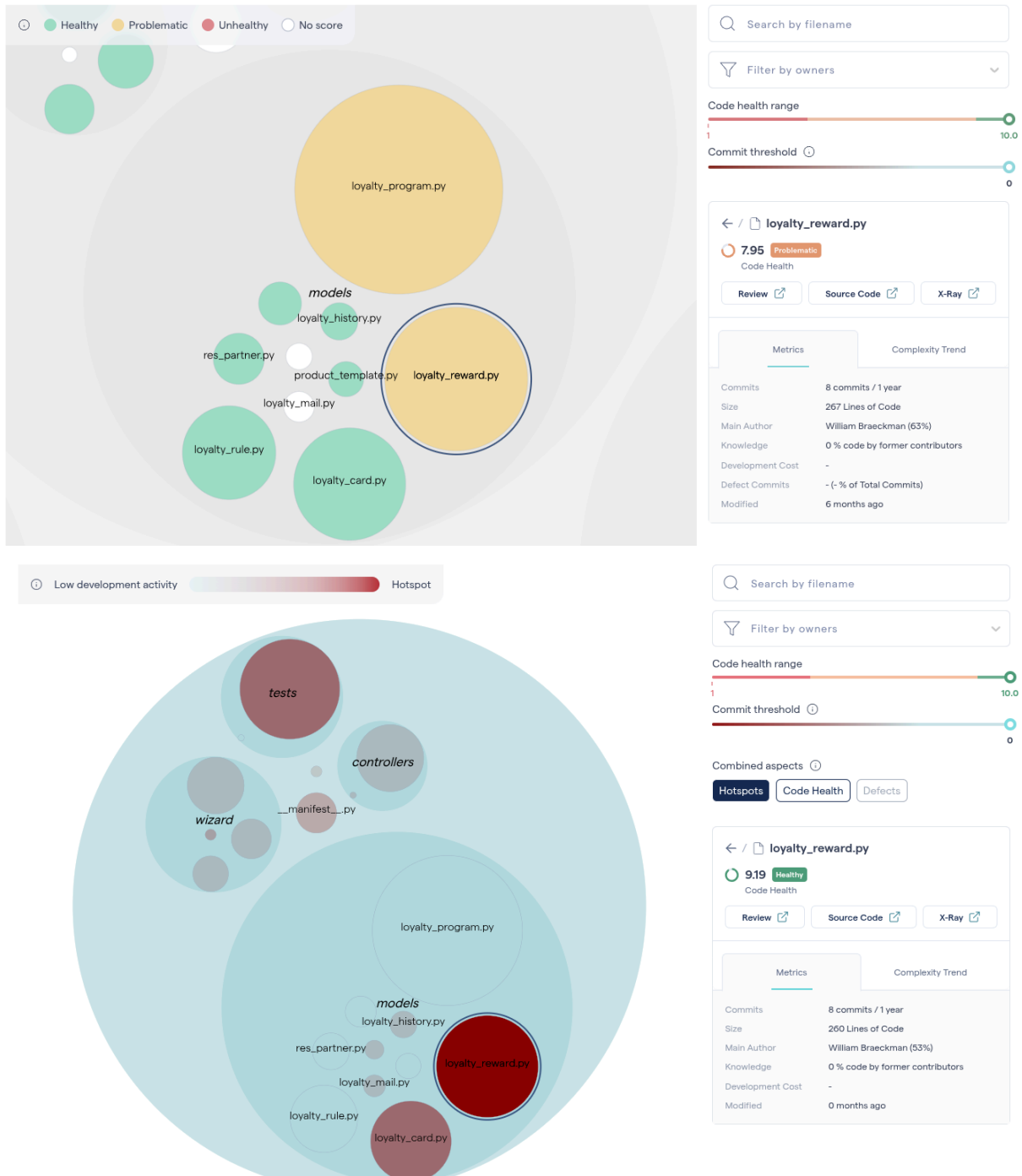


Figure 15. `loyalty_reward.py` health score before and after refactoring, and activity hotspots after beginning our refactoring efforts

Although not completely relevant, it is interesting to observe this script is now marked as a hotspot in CodeScene. That's not surprising, since most of the changes have been made recently by us. This activity can raise the risk score, even if the actual code quality has improved.

These changes made a clear difference. In [Figure 15](#) it can be observed that the CodeScene “Code Health” score for the `loyalty_reward` file went up from **7.95 (problematic)** to **9.19 (healthy)**, showing a noticeable improvement in how maintainable and clean the code is.

We also refactored the `_program_type_default_values` function, which had become quite large, not because of complex logic, but because it was full of hardcoded strings. That made it hard to see what the function was really doing. To make it clearer, we split it into smaller helper functions. This kept the logic the same but made the code easier to read and work with. Breaking it up like this also helps make future changes easier.

Finally, as shown in [Figures 16](#), [17](#) and [18](#), the **loyalty module** overall is now in a good state. CodeScene no longer highlights any complicated or high-risk functions in this part of the codebase. According to the tool, there are currently no major refactoring targets left in the module, which suggests our work has paid off.

After applying all the changes, we ran the full test suite to ensure nothing was broken. Every previously passing test still passed, confirming that the refactoring didn’t introduce any regressions.

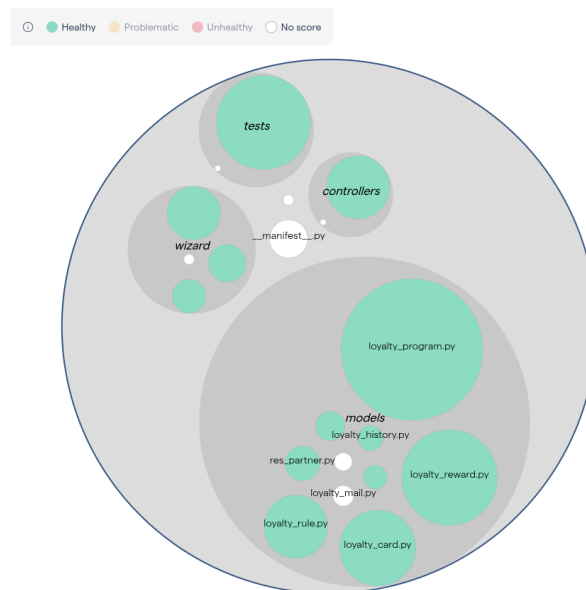


Figure 16. Overview of the loyalty module after our refactoring attempts

Function name	Change Frequency	Lines of Code	Code Review	Function Complexity	Actions
LoyaltyProgram_program_type_default_values	12	13		1	

Figure 17. CodeScene analysis indicating that there are no more complicated functions to refactor

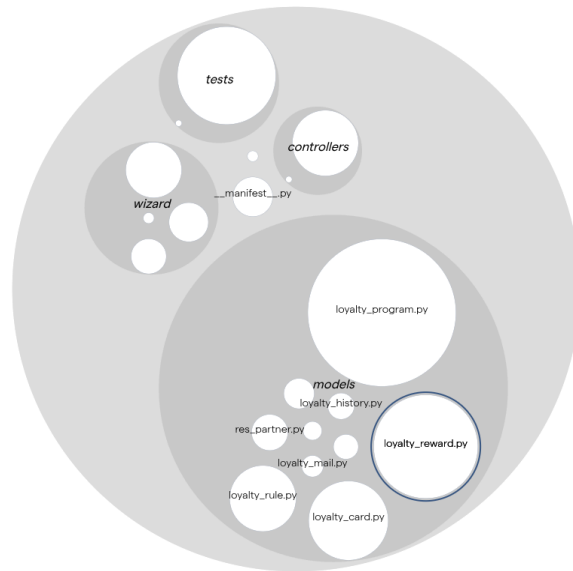


Figure 18. CodeScene showing that there are no refactoring targets inside the loyalty module after our refactoring attempts

4.4 Changes and new additions

Odoo's official documentation and tutorials proved valuable for understanding how the module's logic interacted with the Odoo web interface and database. Specifically, [Chapter 7](#) of the Odoo documentation talks about relations between models.

Since we wanted our changes to be reflected in the web interface in order to simulate a real world use case, we needed to modify two main components of the module: (1) the core logic, which we extended to support the selection of multiple product categories within a loyalty rule, and (2) the user interface, which we adjusted to reflect and accommodate these changes.

As previously portrayed in [Figure 10](#) and described in *Section 3.1*, the original implementation of the module was designed specifically for *Many2one* relations regarding the product categories. This design limited each loyalty rule to only be associated with one product category.

The first step involved modifying the `loyalty_rule.py` file by changing the existing *Many2one* relationship for product categories to a *Many2many* relationship. This change allowed each loyalty rule to be associated with multiple product categories instead of just one.

Since this modification affected how product categories are stored but not how they appear in the user interface, we also needed to update the module's view files to support the new functionality.

After these changes, we tested the updated module by creating a rule with multiple product categories and placing an order that included at least one of those categories. In this scenario, loyalty points were correctly awarded, confirming that the modifications functioned as intended.

At this point, we identified an opportunity to extend the functionality further. We decided to add a toggle option that would change how the selected categories are treated when evaluating an order. The idea was to let the rule specify whether *all* selected categories must be present in order to earn points, or if the presence of *any* one category would be sufficient. To implement this, we introduced a toggle labeled "Require All Categories."

When this option is enabled, the loyalty rule only awards points if the order contains products from *all* selected categories. When the toggle is disabled, the rule is satisfied if the order includes products from *any* of the selected categories.

Open: Conditional rules

CONDITIONS

Minimum Quantity 3

Minimum Purchase 0.00 tax included

AMONG

Products

Required Categories ?

A X B X

Require All Categories ?

<--- New addition

Product Tag

POINT(S)

Grant 1,000.00 Loyalty point(s)

☒ per order

☐ per \$ spent

☐ per unit paid

Save

Discard

Delete

Figure 19. The loyalty rule creation view containing the “Require All Categories” addition

In [Figure 19](#), we show a rule that awards 1.000 loyalty points per order if the order includes at least 3 items from the required Categories A and B. Since the toggle is enabled, both categories must be present in the order for the points to be awarded.

S00690

Customer

Azure Interior
4557 De Silva St
Fremont CA 94538
United States – US12345677

Order Date ?

05/14/2025 11:07:12

Payment Terms

End of Following Month

Quotation Template

Order Lines

Quote Builder

Other Info

Product	Quantity	Delivered	Invoiced	Unit Price	Taxes	Amount
A	2.00	0.00	0.00	10.00		\$ 20.00
B	1.00	0.00	0.00	10.00		\$ 10.00

Add a product
Add a section
Add a note
Catalog

Coupon Code

Reward

Terms and conditions...

Untaxed Amount: \$ 30.00

Total: \$ 30.00

Loyalty Card :

Issued : 1000 Loyalty point(s)

Used : 0 Loyalty point(s)

Figure 20. An example order with 3 items from categories A and B successfully awarding 1.000 loyalty points to the customer based on the established rule

The order shown in [Figure 20](#) is considered valid because it includes at least one product from both Category A and Category B. Specifically, the order contains 2 units of a product from Category A and 1 unit from Category B, meeting the minimum requirement of 3 total items across the required categories. As all conditions are satisfied, the customer is awarded 1.000 loyalty points. If the toggle were disabled, the rule would be less strict, and the points would be issued even if the order included only products from either Category A or B, as long as the total quantity reached at least 3.

In order for this toggle switch to be implemented, we needed to further modify the `loyalty_rule_views.xml` file by adding the toggle switch in the form of a checkbox with all the necessary information that the user should know about its functionality.

To ensure that providers could easily identify whether a rule applies to all or any of the selected product categories, we updated the `loyalty_reward_views.xml` file to display the toggle setting, as shown in [Figure 21](#).

The screenshot displays the 'Rules & Rewards' configuration page for a loyalty program named 'AB'. The page is divided into two main sections: 'Conditional rules' and 'Rewards'.

Program Details:

- Program Name:** AB
- Program Type:** Loyalty Cards
- Currency:** USD
- Points Unit:** Loyalty point(s)
- Start Date:** ?
- End date:** ?
- Limit Usage:** ☐
- Available On:** ☒ Sales

Conditional rules:

- Rule 1:** Among: A B (all required) 10.00 Loyalty point(s) per order
- Rule 2:** Among: A B (any) 10.00 Loyalty point(s) per order

Rewards:

- Reward 1:** 5.00% discount on your order
- Reward 2:** In exchange of 200.00 Loyalty point(s)

Figure 21: Indication whether a rule requires all or any of the product categories

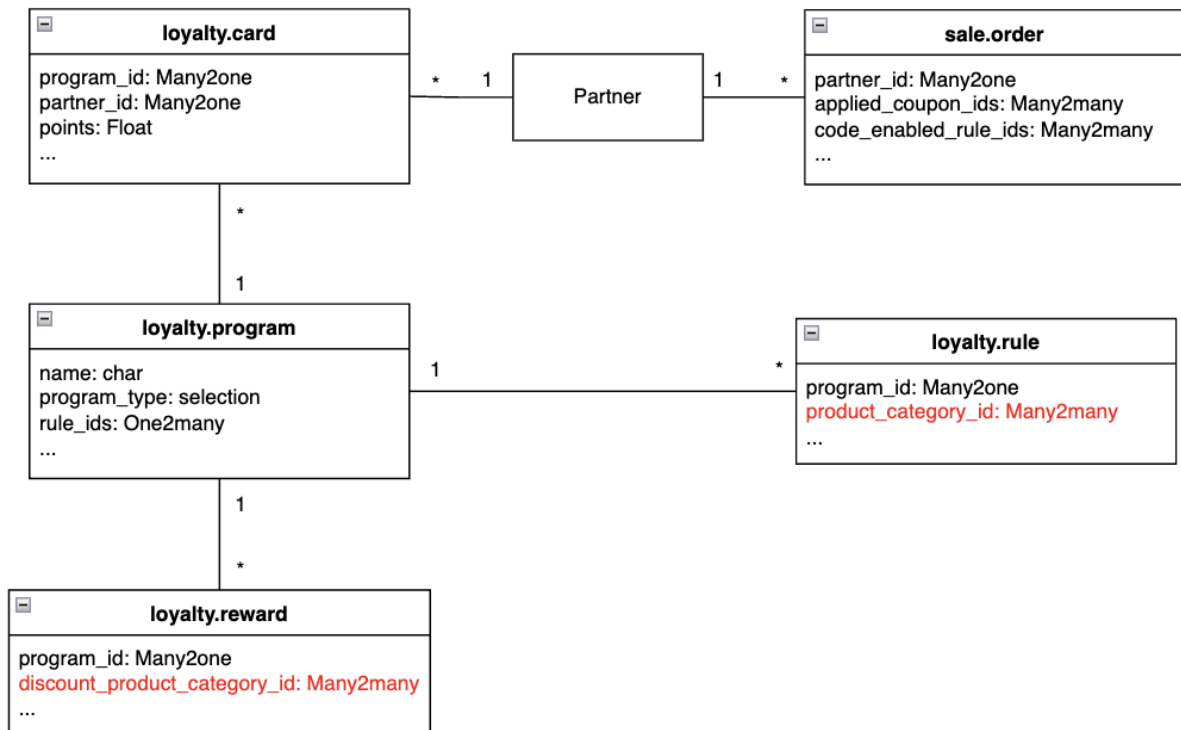


Figure 22: Updated UML diagram, reflecting the changes made

[Figure 22](#) presents the updated UML diagram, reflecting the changes made to the module's data model; the modified parts are highlighted in red.

4.5 Integration with existing database

From early on, one of our biggest concerns was how we would safely modify important parts of the database structure, such as the database tables, and how these modifications would affect the rest of the addons.

After going through the documentation, we quickly understood that Odoo is designed in such a way that all database relationships and table fields are defined from within the codebase, rather than being defined from the database itself. When loading up Odoo, the system checks for the defined relationships and builds up the table structure according to the needs of the code. This assures a correct migration and a cascading update effect when including modifications

During development, the codebase was tested in two different ways. Firstly we explored manually the web interface and the testing suite available (refer to section [x](#)). Given that our end goal was centered around the interface usage, being able to have a clear, visual feedback of our applied changes was necessary.

The development process involved iterative testing through both the web interface and the existing test suite. Changes were reverted as needed when issues were encountered. This cycle continued until the intended functionality was achieved and all original tests passed successfully. Additional tests were implemented to verify the new features. These tests focused on confirming that all selected product categories were correctly recorded in the database and that loyalty points were awarded as expected when a valid order was created.

5. Conclusions

In this project, we focused on reengineering the loyalty module within the Odoo codebase. We identified a **key limitation in its existing functionality**, as well as opportunities to improve the structure and clarity of the module's code. These factors made it a suitable candidate for reengineering.

Our modifications were implemented with care to avoid disrupting other components of the Odoo system. **We validated the changes through the existing test suite and conducted extensive manual testing using the web interface.** The entire process was done in accordance to conventional reengineering patterns, which helped keep focused on the end goal without our attention being diverted elsewhere.

Overall, the reengineered module introduces enhanced functionality compared to the original version, while maintaining consistency with the existing codebase and user experience.

While our changes are limited in scope, they demonstrate the potential for further development of the loyalty module. Future improvements could continue building on this foundation to expand its capabilities.